

# Prédiction et Optimisation distribuée et intelligente de ressources dans le Network Slicing 5G

Nadir NEHILI  
Nahil EL BEZZARI  
Yassine LAZIZI

CY Cergy Paris Université  
**CY Tech école d'ingénieurs**  
Cergy, France



28 avril 2026

## Résumé

Dans le contexte du découpage réseau (*Network Slicing*) 5G/6G, la gestion proactive des ressources radio face aux changements du trafic constitue un défi technique majeur. Pour y répondre, ce travail propose un contrôleur réseau intelligent en boucle fermée, associant un modèle de prédiction (LSTM avec mécanisme d'attention) et un moteur d'optimisation mathématique (MILP). Le problème d'allocation est modélisé via une fonction de coût pénalisant conjointement la dégradation du débit et la probabilité de perte de paquets.

Les évaluations *offline* révèlent que cette synergie permet un gain d'optimisation global de près de **40%** par rapport à une allocation statique inter-tranches. Afin de valider ces performances hors des environnements purement synthétiques, nous testons ensuite notre approche sur un banc d'essai réaliste basé sur le simulateur **Simu5G**, un simulateur basé sur OMNeT++. Soumis à des scénarios de trafic hétérogènes répartis sur plusieurs stations de base, le système démontre l'avantage crucial de l'anticipation temporelle face aux micro-rafales de trafic. Les évaluations en temps réel confirment notre approche : le contrôleur maintient la stabilité globale du réseau et réduit significativement les violations des accords de niveau de service (SLA), marquant notamment une diminution de **17%** des violations pour le trafic critique URLLC par rapport aux approches réactives traditionnelles.

**Mots-clés :** Network Slicing, 5G, Allocation proactive, Deep Learning, LSTM, Optimisation MILP, Qualité de Service (QoS), Simu5G, OMNeT++.

# Table des matières

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>3</b>  |
| <b>2</b> | <b>Travaux existants</b>                                   | <b>3</b>  |
| 2.1      | Modèles de prédiction                                      | 3         |
| 2.2      | Modèles d'allocation                                       | 3         |
| 2.2.1    | Approches mathématiques, heuristiques et règles explicites | 3         |
| 2.2.2    | Apprentissage par renforcement (DRL)                       | 4         |
| 2.3      | Positionnement                                             | 5         |
| <b>3</b> | <b>Infrastructure réseau</b>                               | <b>5</b>  |
| 3.1      | Architecture simulée                                       | 5         |
| 3.2      | Slices et SLA                                              | 5         |
| <b>4</b> | <b>Données</b>                                             | <b>6</b>  |
| 4.1      | Scénarios d'usage issus de la simulation                   | 6         |
| 4.2      | Enrichissement par scénarios synthétiques                  | 7         |
| <b>5</b> | <b>Algorithme</b>                                          | <b>8</b>  |
| 5.1      | Prédiction du trafic                                       | 9         |
| 5.1.1    | Préparation des variables d'entrée                         | 9         |
| 5.1.2    | Architecture du modèle                                     | 9         |
| 5.1.3    | Entraînement                                               | 10        |
| 5.2      | Réallocation des ressources                                | 11        |
| 5.2.1    | Signal de violation SLA : deux approches selon le contexte | 12        |
| 5.2.2    | Définition et estimation du taux de charge $\rho$          | 12        |
| 5.2.3    | Calibration de la sigmoïde $p_{loss}(\rho)$                | 13        |
| 5.2.4    | Estimation de $\hat{\rho}$ dans la boucle fermée           | 13        |
| 5.2.5    | Fonctions de coût et formulation MILP                      | 13        |
| <b>6</b> | <b>Résultats</b>                                           | <b>14</b> |
| 6.1      | Protocole d'évaluation offline                             | 14        |
| 6.2      | Résultats offline                                          | 15        |
| 6.2.1    | Vue d'ensemble comparative                                 | 15        |
| 6.2.2    | Analyse par station de base                                | 15        |
| 6.3      | Apprentissage en continu                                   | 16        |
| 6.4      | Validation live sur Simu5G                                 | 16        |
| <b>7</b> | <b>Conclusion et perspectives</b>                          | <b>17</b> |
| 7.1      | Synthèse des contributions                                 | 17        |
| 7.2      | Limites et discussion critique                             | 18        |
| 7.3      | Perspectives                                               | 18        |

# 1 Introduction

L'avènement des réseaux 5G et de la future 6G repose fondamentalement sur le paradigme du (*Network Slicing*). Cette technologie permet de déployer simultanément de multiples services aux exigences hétérogènes, tels que le haut débit mobile (eMBB), les communications ultra-fiables à faible latence (URLLC), ou l'Internet des objets massif (mMTC), sur une infrastructure physique partagée. Cependant, la gestion dynamique de l'allocation des ressources radio face aux changements imprévisibles du trafic constitue un défi technique majeur.

Si les solutions historiques de gestion des ressources sont majoritairement réactives, se contentant de corriger les dégradations *a posteriori*, elles montrent aujourd'hui leurs limites.

Dans un environnement réseau complexe, caractérisé par des stations de base aux profils hétérogènes et des micro-rafales de trafic soudaines, une approche réactive ne peut empêcher la violation des Services Level Agreements (SLA). Pour pallier ce problème, la littérature scientifique explore de plus en plus les pistes de l'anticipation du trafic et de la réallocation préventive. Toutefois, comme nous l'analysons dans la section suivante, ces travaux abordent le plus souvent la prédiction et l'allocation comme des problèmes isolés, sans véritable couplage dynamique en boucle fermée.

## 2 Travaux existants

### 2.1 Modèles de prédiction

La prédiction du trafic dans le contexte du *network slicing* a vu l'émergence d'architectures hybrides visant à capturer des dépendances spatio-temporelles complexes. [11] propose un pipeline combinant convolutions 1D, LSTM et Transformer pour gérer différents horizons temporels. Néanmoins, Tuna et al. [13] nuancent l'apport de l'attention multi-tête en démontrant que si les Transformers excellent sur le long terme, les modèles LSTM restent plus précis sur des horizons courts (1 à 4 heures). Z. Min et al. [7] intègrent quant à eux un pure Transformer (TST) au sein d'une boucle proactive de Jumeau Numérique, bien que l'efficacité du système repose entièrement sur la fidélité de ce modèle virtuel. Sur le plan spatial, afin de dépasser les limites des projections géographiques en grille 2D, [15] introduit *MRSTCGN*. Cette approche exploite des convolutions sur graphes (GCN) à représentations multiples pour capturer l'hétérogénéité des liens et la topologie logique du réseau, bien qu'elle reste validée sur des proxys de trafic routier.

Au-delà du choix architectural, la nature des données d'entrée et la fonction d'optimisation s'avèrent déterminantes. Pour s'affranchir des limites des séries temporelles univariées, le modèle *LSTM-FSD* [12] intègre un vecteur d'état multivarié (charge, type de tranche) couplé à un solveur analytique post-prédiction (*LP-KPI*) permettant d'estimer directement la latence et les pertes de paquets. Concernant les fonctions de perte, on remarque que les fonctions standards (MSE, MAE) étant symétriques, échouent à refléter le coût critique d'un sous-dimensionnement des ressources. Pour y remédier, [13] propose une fonction asymétrique (*wMAE*) pénalisant plus lourdement les sous-estimations de trafic, alignant ainsi mathématiquement l'entraînement du modèle avec l'objectif de prévention des violations SLA.

Enfin, face aux défis de passage à l'échelle et d'hétérogénéité, les approches centralisées montrent leurs limites. N. Pavlidis et al. [9] démontrent la viabilité de l'apprentissage fédéré (*FedAvg*) appliqué à des LSTM locaux. En évitant la centralisation des données brutes, leur approche réduit drastiquement l'empreinte énergétique et les communications, tout en modélisant avec une bien meilleure précision les comportements de trafic locaux atypiques (stades, gares) qui sont souvent gommés par les modèles globaux.

### 2.2 Modèles d'allocation

#### 2.2.1 Approches mathématiques, heuristiques et règles explicites

Un premier axe de la littérature se concentre sur l'optimisation mathématique de l'allocation des ressources. Pour s'affranchir des limitations du routage mono-chemin, [4] modélise l'allocation comme un problème non-linéaire (MINLP), ultérieurement relaxé et résolu via un algorithme de Génération de Colonnes (cCG). Bien que cette méthode offre des solutions quasi-optimales (gap < 1%) sur des topologies réalistes, son application isolée sur un instantané figé de la demande l'empêche de s'adapter aux fluctuations soudaines du trafic. Face à cette rigidité, des approches méta-heuristiques ont émergé, privilégiant la vitesse d'exécution. Par exemple, [14] propose l'algorithme hybride *IavgOA*, combinant l'exploration (inspirée du vol des vautours) et l'exploitation (inspirée des sauterelles). En évaluant simultanément

le coût de communication, la QoS et la consommation énergétique, cette approche réduit le temps de calcul jusqu'à 86% par rapport aux algorithmes génétiques classiques, bien qu'elle ne garantisse aucune convergence vers l'optimum global.

Parallèlement, la nécessité d'opérer avec des processus décisionnels auditables, par opposition aux « boîtes noires » des réseaux de neurones, a motivé des approches à base de règles. Le framework *CARO* [2] structure l'orchestration en trois modules (SDN/NFV) pilotés par des règles proportionnelles et des pénalités préventives basées sur des lissages exponentiels. Bien qu'elle garantisse une totale transparence des décisions d'équilibrage de charge, cette méthode est figée par des seuils prédéfinis par l'opérateur et manque de capacités d'apprentissage. Pour pallier l'incapacité d'adaptation des règles statiques et la rigidité des modèles exacts, des frameworks comme *EvoNetSlice* [6] ont couplé des algorithmes évolutionnaires à une analyse IA pour ajuster dynamiquement les poids des objectifs d'allocation (débit, latence, pertes). Néanmoins, ces approches restent souvent confinées à des environnements purement synthétiques.

En résumé, ce pan de la littérature peine à concilier trois objectifs fondamentaux : l'optimalité mathématique, l'adaptabilité dynamique et le passage à l'échelle en environnement réel.

### 2.2.2 Apprentissage par renforcement (DRL)

L'apprentissage par renforcement profond (DRL) s'impose comme l'alternative majeure pour apprendre des politiques de réallocation en interaction avec le réseau. [3] propose *PW-DRL*, une architecture centralisée combinant l'algorithme TRPO pour les décisions d'admission et l'optimisation de Lyapunov pour la satisfaction des contraintes de long terme (puissance, capacité). Si le couplage avec un module de prédiction auxiliaire améliore significativement la stabilité du système, l'approche peine à sécuriser les contraintes instantanées critiques pour les services à faible latence.

Cette question de la sûreté des décisions en cours de déploiement est traitée par [8] via le framework *SafeSlice*. Conçu pour les environnements O-RAN, il modélise le problème sous forme de processus décisionnel de Markov contraint (CMDP) et introduit une couche de sûreté (*safety layer*). Avant l'application d'une action, cette couche interroge un régresseur (XGBoost) prédisant la latence résultante ; en cas de violation projetée, l'action est corrigée vers l'allocation faisable la plus proche. Bien que cette projection réduise drastiquement les violations instantanées (de plus de 93%), la sécurité totale du système reste ultimement dépendante de la précision du modèle de régression sous-jacent.

Enfin, pour surmonter les problèmes de scalabilité et les conflits de ressources inhérents aux agents centralisés, la recherche s'oriente vers le DRL multi-agents (MADRL). Abiko et al. [1] proposent une architecture distribuée basée sur Ape-X où chaque agent contrôle indépendamment une tranche, coordonné basiquement par un arbitre central. Toutefois, pour résoudre les conflits inter-tranches sans signalisation préétablie, [10] introduit *STEP*. En intégrant un goulot d'étranglement de l'information (Information Bottleneck) inspiré des autoencodeurs variationnels, les agents développent de manière autonome un protocole de communication concis. Les résultats démontrent que l'émergence de ce protocole réduit drastiquement les conflits d'allocation et la latence, confirmant le potentiel des architectures MADRL collaboratives.

L'analyse croisée de la littérature fait émerger un double constat qui définit le verrou technologique actuel. D'une part, la prédiction de trafic et l'allocation de ressources sont majoritairement traitées de manière découplée. D'autre part, au sein même des moteurs d'allocation, un compromis persistant oppose les méthodes exactes [4], garantissant l'optimalité mais inadaptées aux fluctuations dynamiques, aux approches heuristiques ou par apprentissage par renforcement [14, 8], offrant une meilleure réactivité mais dépourvues de garanties formelles de convergence vers une solution optimale. D'autant plus, qu'aucun des travaux étudiés ne parvient à coupler véritablement un modèle prédictif intégrant nativement les contraintes SLA à un moteur d'allocation au sein d'une boucle fermée opérationnelle.

C'est précisément dans cet spectre (*research gap*) que s'inscrit notre contribution. Plutôt que de sacrifier l'optimalité mathématique au profit d'heuristiques incertaines, ou de subir la rigidité des approches statiques, notre approche propose de fusionner ces deux paradigmes. Nous introduisons une architecture hybride intégrant un solveur exact (MILP) au sein d'une boucle de contrôle continue, alimentée dynamiquement par un modèle d'apprentissage profond (LSTM) anticipant l'état du réseau. Cette conception permet au système de conserver les garanties d'une optimisation stricte tout en acquérant l'adaptabilité proactive nécessaire pour prévenir les violations de SLA face au sein du trafic.

### 2.3 Positionnement

Par rapport aux approches de prédiction seule (ConvLSTMTransNet [11], LSTM-FSD [12]), notre travail couple le modèle prédictif à un moteur d'allocation opérationnel. LSTM-FSD propose bien un solveur post-prédiction (LP-KPI), mais celui-ci opère sur des KPIs observés et non prédits, maintenant un fonctionnement réactif.

Par rapport aux approches DRL (SafeSlice [8], Ape-X [1], STEP [10]), notre solveur MILP offre des garanties formelles d'optimalité sur la fenêtre courante. SafeSlice utilise un prédicteur statique (XGBoost) sans mise à jour en ligne ; notre apprentissage incrémental permet une adaptation continue.

Par rapport aux méthodes mathématiques pures (MINLP + cCG [4]), notre MILP est résolu en  $< 5$  ms contre plusieurs secondes pour la génération de colonnes, condition nécessaire à une intégration temps réel.

Notre protocole FedAvg LoGO s'inscrit dans la lignée de Pavlidis et al. [9], avec l'extension LoGO apportant un cadre d'évaluation plus rigoureux de la généralisation inter-sites.

## 3 Infrastructure réseau

Pour évaluer notre approche d'orchestration dans des conditions réalistes, nous avons modélisé une infrastructure hétérogène à l'aide du simulateur événementiel OMNeT++ couplé au framework Simu5G. Ce choix technologique nous permet de configurer sur mesure la pile protocolaire 5G, de la couche d'accès au médium (MAC) jusqu'à la couche application, afin d'extraire des métriques de trafic avec une haute précision temporelle.

### 3.1 Architecture simulée

Afin de reproduire la diversité d'un réseau réel et de générer des distributions de données non-identiquement distribuées (non-IID), un défi majeur pour l'apprentissage automatique, nous avons spécifiquement défini et paramétré trois stations de base (gNB) aux caractéristiques matérielles et logicielles distinctes dans notre environnement de simulation.

Ces trois stations ont été configurées de la manière suivante :

- **gNB Macro** : Représente une cellule urbaine macro (UMa) couvrant une vaste zone résidentielle. Nous lui avons alloué un budget radio important de 50 blocs de ressources (Resource Blocks - RBs) et configuré son allocation via l'ordonnanceur *LteMaxCi*.
- **gNB Commerce** : Modélise une micro-cellule urbaine (UMi) déployée dans un centre commercial. Nous avons restreint son budget à 35 RBs, géré par l'ordonnanceur *Proportional Fair* (LtePf).
- **gNB Industrie** : Correspond à une micro-cellule (UMi) déployée en zone industrielle. Pour simuler un environnement fortement contraint, nous l'avons limitée à un budget de seulement 25 RBs, également régi par l'ordonnanceur *LtePf*.

| Station de base | Type d'environnement     | Budget Radio (RBs) | Ordonnanceur |
|-----------------|--------------------------|--------------------|--------------|
| Macro           | UMa (Zone résidentielle) | 50                 | LteMaxCi     |
| Commerce        | UMi (Centre commercial)  | 35                 | LtePf        |
| Industrie       | UMi (Zone industrielle)  | 25                 | LtePf        |

TABLE 1 – Configurations des stations de base définies dans le simulateur.

### 3.2 Slices et SLA

Sur cette infrastructure physique partagée, nous avons configuré le découpage logique du réseau en trois tranches (slices) standardisées. Pour chacune d'elles, nous avons défini dans Simu5G des accords de niveau de service (SLA) précis, en fixant nos propres seuils de latence maximale et de débit minimal afin de refléter fidèlement leurs cas d'usage respectifs :

- **eMBB (Enhanced Mobile Broadband)** : Dédicée aux applications très gourmandes en bande passante (ex. streaming vidéo HD), nous avons fixé son exigence de débit minimum à 10 Mbps et paramétré sa tolérance à la latence à 60 ms.

- **URLLC (Ultra-Reliable Low-Latency Communication)** : Destinée aux applications critiques (ex. robotique industrielle, téléchirurgie), nous lui avons imposé une contrainte de latence extrêmement sévère de 25 ms, associée à un besoin en débit plus modeste défini à 0,5 Mbps.
- **mMTC (Massive Machine Type Communications)** : Déployée pour la collecte de données issues de capteurs IoT, nous n'avons configuré aucune exigence de débit minimal garanti pour cette tranche, et défini une tolérance à la latence plus souple allant jusqu'à 300 ms.

| Tranche (Slice) | Cas d'usage type        | Latence max. configurée | Débit min. configuré |
|-----------------|-------------------------|-------------------------|----------------------|
| eMBB            | Streaming vidéo HD      | 60 ms                   | 10 Mbps              |
| URLLC           | Robotique, télémedecine | 25 ms                   | 0,5 Mbps             |
| mMTC            | Capteurs IoT            | 300 ms                  | —                    |

TABLE 2 – Seuils SLA paramétrés par tranche dans l'environnement Simu5G.

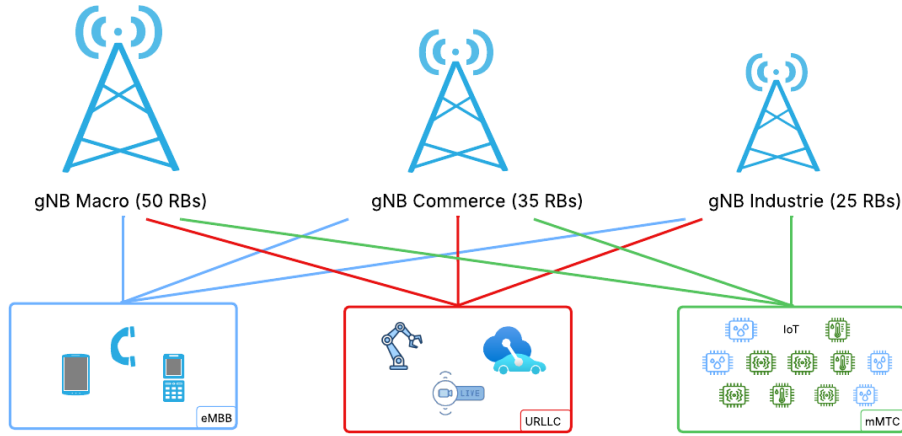


FIGURE 1 – Représentation de l'architecture réseau simulée sur Simu5G.

## 4 Données

Pour entraîner notre modèle d'apprentissage profond et valider notre algorithme d'allocation en boucle fermée, la disponibilité de données qualifiées est indispensable. L'acquisition de telles données constitue cependant un défi majeur, les traces de trafic réelles étant souvent protégées par le secret industriel des opérateurs télécoms. De plus, les rares jeux de données publics peinent à être exploités dans notre contexte : ils omettent fréquemment la segmentation par tranches ou manquent cruellement de granularité temporelle. Surtout, même lorsque l'on dispose d'un jeu de données réel d'excellente qualité technique, à l'instar du dataset [5] horodaté à la milliseconde, son contenu capture presque exclusivement un réseau opérant en régime « normal ». L'absence de variations extrêmes de trafic, de congestions sévères ou d'anomalies limite grandement son intérêt pour entraîner un modèle prédictif censé apprendre à réagir face à des cas critiques très variés. Face à ce constat, nous avons pris le parti de générer notre propre jeu de données directement depuis notre infrastructure Simu5G. Ce processus de constitution s'est déroulé en deux étapes : la collecte de métriques issues de scénarios d'usage simulés, suivie d'un enrichissement par la génération de transitions synthétiques.

### 4.1 Scénarios d'usage issus de la simulation

La première brique de notre jeu de données (`all_simu5g.csv`) a été acquise en simulant le réseau sur OMNeT++. Ce jeu de base comprend 183 812 enregistrements, correspondant chacun à l'état du réseau pour une seconde de simulation précise, croisant une tranche réseau et une station de base spécifiques.

Pour chaque pas de temps, l'environnement simule et capture une dizaine de caractéristiques clés, dont :

- **Le contexte** : Horodatage (`Time_Sec`), type de tranche (`Slice_Type`), identifiant de la station (`gNB_id`) et nom du scénario.
- **Les performances brutes (KPIs)** : Le débit effectif en Mbps, la latence en millisecondes, le pourcentage de paquets perdus et la charge globale du réseau.
- **L'état vis-à-vis des SLA** : Une variable binaire indiquant si le SLA est respecté à l'instant présent (`SLA_OK`), ainsi qu'une variable cible essentielle pour l'apprentissage, indiquant si une violation va survenir dans la fenêtre future (`SLA_Violated_in_15s`).

Afin de garantir la robustesse du futur modèle, nous avons simulé **12 scénarios** très contrastés. Ces derniers couvrent un spectre large de situations opérationnelles : des états calmes (trafic de nuit), des montées en charge modérées ciblées sur certaines tranches, mais également des événements critiques tels que des pics massifs d'utilisateurs eMBB, des saturations globales de l'infrastructure, ou encore des pannes partielles provoquant des redirections soudaines de trafic. Il est fondamental de souligner que ces configurations ne sont pas arbitraires : elles sont rigoureusement fondées sur des incidents réseaux authentiques et des situations réalistes documentées par les opérateurs (à l'image de la panne historique de KDDI ou des pics de charge liés à des événements comme la Coupe du Monde de la FIFA), assurant ainsi la pertinence opérationnelle de notre base d'apprentissage.

## 4.2 Enrichissement par scénarios synthétiques

Bien que couvrant des situations de charge extrêmes, l'analyse du dataset de base a révélé une limitation fonctionnelle majeure pour l'apprentissage prédictif. Les scénarios réels génèrent principalement des états stables prolongés, le réseau est soit durablement calme, soit durablement saturé. Or, l'objectif d'un contrôleur proactif est d'anticiper la saturation, le modèle LSTM a donc un besoin impératif d'apprendre sur des exemples montrant des augmentations progressives de la charge.

Pour pallier ce manque d'états transitoires, nous avons développé un générateur de scénarios synthétiques (`create_rampup.py`). Ce script a produit **14 scénarios** supplémentaires par interpolation non-linéaire entre un scénario de base (calme) et un scénario cible (saturé). Afin de modéliser une transition réaliste, nous avons utilisé une **interpolation sigmoïde** définie comme suit :

$$KPI(t) = (1 - \alpha(t)) \cdot KPI_{LOW}(t) + \alpha(t) \cdot KPI_{HIGH}(t) + \text{bruit} \quad (1)$$

Où  $\alpha(t)$  est une fonction évoluant linéairement de 0 à 1 durant la phase de transition, et où un bruit gaussien de 2% est ajouté pour conserver la volatilité naturelle des métriques télécoms.

Cette méthodologie a permis de concevoir cinq profils de transition dynamiques par tranche :

- **Trans\_RampUp / RampDown** : Des phases massives de montée ou de descente en charge (représentant 40% de la durée du scénario) encadrées par des régimes stables.
- **Trans\_Peak** : L'apparition et la disparition d'un pic court et intense (20% de la durée).
- **Trans\_Gradual** : Une dégradation lente et continue du trafic.
- **Trans\_LowViol** : De légères fluctuations provoquant des violations éparses, forçant le modèle à affiner sa sensibilité.

L'ajout de ces 71 976 lignes de transitions a permis d'aboutir à un jeu de données final robuste (`all_simu5g_trans.csv`) totalisant **255 812 lignes** réparties sur **26 scénarios**. Ce dataset enrichi constitue un support d'apprentissage idéal, permettant au réseau de neurones de ne pas simplement réagir aux états de congestion, mais d'en identifier formellement les signes temporels avant-coureurs.

L'intérêt d'une gestion intelligente prend tout son sens face aux limites des politiques d'allocation traditionnelles. Sur notre architecture, l'application d'une allocation statique par défaut (attribuant un nombre fixe de RBs à chaque tranche indépendamment de la demande instantanée) conduit à des dégradations massives des services critiques. En effet, sous un régime statique, l'environnement simulé révèle que la tranche eMBB subit une violation de ses SLA dans 48% des cas, tandis que la tranche URLLC, pénalisée par sa forte sensibilité aux micro-rafales, est en violation dans 61% des cas. Ce constat met en évidence l'incapacité d'un paramétrage rigide à garantir les exigences de QoS, et justifie la nécessité d'un contrôleur dynamique capable de réallouer les ressources radio de manière proactive.

## 5 Algorithme

L'objectif est ici de permettre l'allocation dynamique de ressources au sein d'un réseau afin de garantir une qualité de service (QoS) minimale pour chaque tranche réseau. Ainsi, on accorde une attention particulière au respect continu des *SLA*. L'idée est donc de mener une démarche prédictive sur l'évolution du réseau afin de permettre une réallocation des ressources en amont. Voici donc la démarche algorithmique que l'on suit :

- **Récupération des données** : Les paquets envoyés au sein de la simulation réseau lors des  $X$  dernières secondes sont récupérés.
- **Prédiction des données** : Ces paquets récupérés sont ensuite transmis au modèle afin de prédire les paquets qui seront créés et envoyés durant les  $Y$  prochaines secondes de la simulation réseau.
- **Vérification** : Les paquets prédits sont ensuite analysés afin de s'assurer qu'ils respectent les *SLA* fixés. En fonction de la conformité aux *SLA*, les ressources nécessaires sont allouées afin d'éviter que ces paquets ne respectent pas les *SLA*, et par extension garantir une QoS satisfaisante.
- **Répétition** : Ces étapes sont répétées toutes les  $Z$  secondes afin de monitorer continuellement l'évolution du réseau.

Notons que les hyperparamètres  $X$ ,  $Y$  et  $Z$  influent grandement la pertinence ainsi que les performances de cet algorithme.

Faire varier le nombre de secondes récupérées ( $X$ ) peut rallonger le temps nécessaire au modèle afin de réaliser la prédiction, cependant une quantité minimale de données est tout de même requise afin de réaliser une prédiction correcte.

La prédiction des paquets qui seront envoyés ( $Y$ ) nous expose à une erreur de plus en plus importante à mesure que l'on se projette dans le futur. Pour autant, il apparaît comme nécessaire de ne pas trop diminuer ce paramètre afin d'éviter une répétition trop fréquente de l'algorithme ( $Z$ ) qui serait coûteuse en ressources de calcul et pourrait ne pas laisser le temps nécessaire à la prédiction ainsi qu'à l'allocation. Dans notre implémentation finale, nous fixons  $X = 60$  s,  $Y = 15$  s et  $Z = 15$  s.

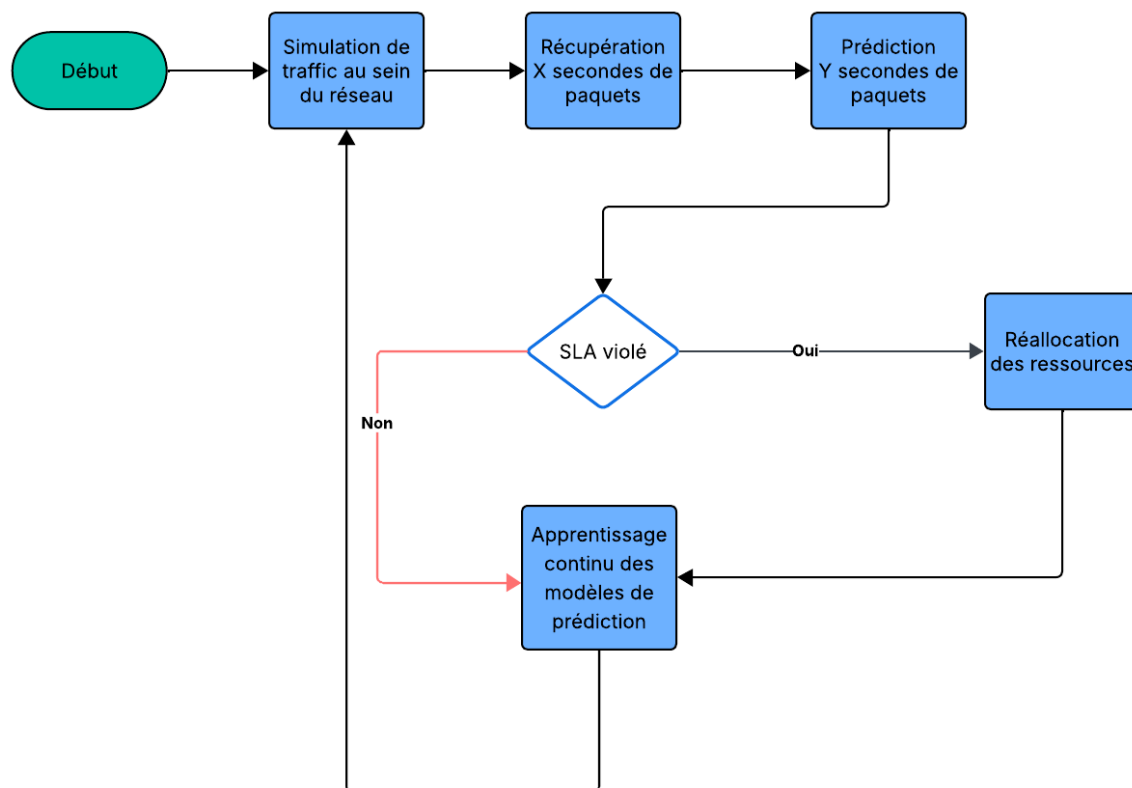


FIGURE 2 – Algorithme de prédiction et réallocation dynamique.



## 5.1 Prédiction du trafic

Pour répondre aux exigences de prédiction proactive des indicateurs de performance (KPIs) des slices, nous avons conçu une architecture Seq2Seq basée sur des LSTM couplés à un mécanisme d'attention. Pour chacun des 3 slices (eMBB, URLLC, mMTC) un modèle global est entraîné par apprentissage fédéré avec les sous-modèles capturant les caractéristiques propres à chaque gNB.

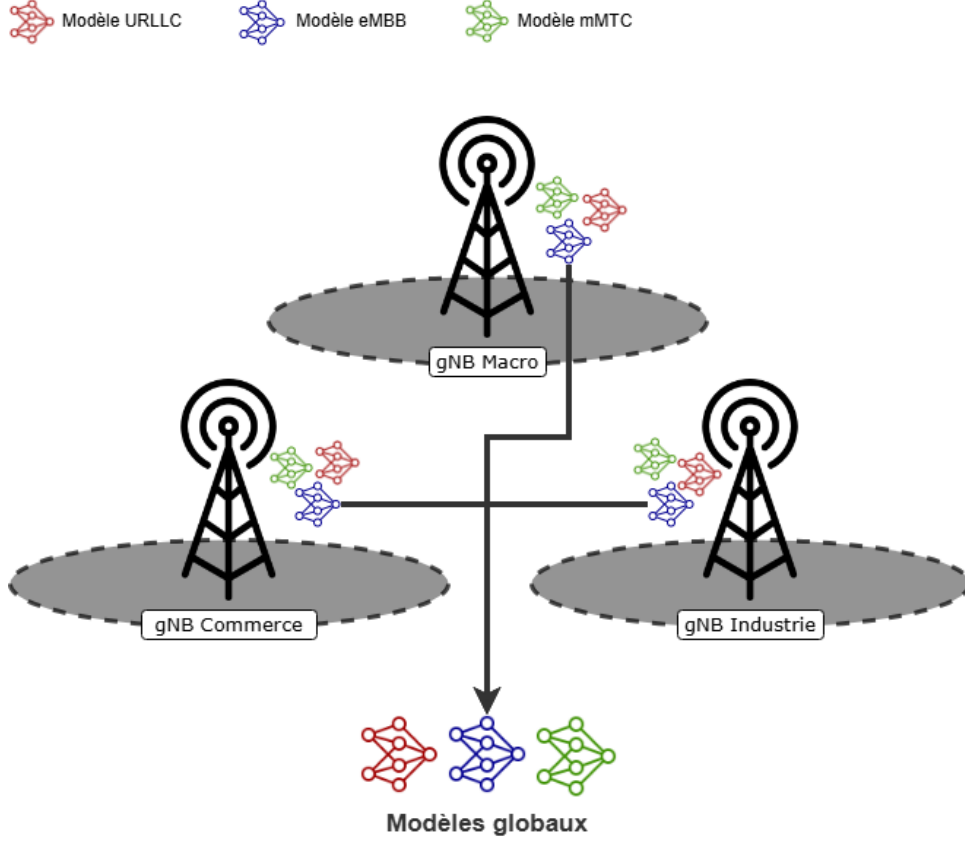


FIGURE 3 – Vue d'ensemble de la prédiction du trafic selon l'architecture réseau simulée.

### 5.1.1 Préparation des variables d'entrée

Chaque pas de temps de la fenêtre d'observation est représenté par un vecteur de **15 caractéristiques** organisées en trois groupes :

1. **KPIs bruts** (5 features) : débit (`throughput`), latence log-normalisée (`latency_log1p`), taux de perte effective, jitter log-normalisé (`jitter_log1p`) et charge réseau (`network_load`).
2. **Tendances** (5 features) : différences premières de chacune des 5 variables brutes, capturant la direction et la vitesse d'évolution instantanée.
3. **Volatilités** (5 features) : écarts-types glissants sur une fenêtre de 5 secondes, codant la turbulence locale du signal.

Lors des épisodes de congestion, l'absence de mécanisme de rejet (*packet drop*) pour les paquets en attente prolongée dans les files de Simu5G entraîne une accumulation continue. Par conséquent, les métriques de latence et de jitter ont tendance à croître indéfiniment vers des valeurs extrêmes, ce qui fausse l'apprentissage du modèle. L'utilisation de transformations logarithmiques (`log1p`) permet d'écraser mathématiquement cette croissance artificielle pour stabiliser la distribution, tout en évitant d'introduire des valeurs infinies lorsque la métrique est nulle.

### 5.1.2 Architecture du modèle

Le cœur prédictif du contrôleur repose sur une architecture neuronale de type *Sequence-to-Sequence* (Seq2Seq), adaptée à la modélisation des dynamiques temporelles complexes du trafic réseau. Ce type

d'architecture permet d'apprendre une transformation entre une fenêtre d'observation historique et un horizon futur de prédiction.

L'architecture globale est composée d'un encodeur et d'un décodeur LSTM multi-couches. Chaque module comporte trois couches cachées de 256 neurones, avec un taux de *dropout* fixé à 0,25 afin de limiter le surapprentissage tout en conservant une capacité d'abstraction élevée.

**Encodeur :** L'encodeur traite la séquence historique correspondant à une fenêtre d'observation de 60 secondes. Il repose sur une architecture LSTM bidirectionnelle, permettant de capturer simultanément les dépendances temporelles ascendantes et descendantes au sein de la fenêtre d'entrée.

Cette bidirectionnalité améliore la représentation contextuelle globale en intégrant l'évolution complète du trafic sur l'intervalle considéré, ce qui est particulièrement utile dans les environnements réseau où les variations de charge peuvent être rapides et non linéaires.

**Mécanisme d'attention :** Afin d'améliorer la capacité du modèle à identifier les instants les plus informatifs dans l'historique, un mécanisme d'attention additive (attention de Bahdanau) est intégré entre l'encodeur et le décodeur.

À chaque pas de temps du décodeur, un vecteur de contexte dynamique  $c_t$  est calculé à partir des états cachés de l'encodeur  $h_i^{enc}$  et de l'état précédent du décodeur  $s_{t-1}^{dec}$  :

$$e_{t,i} = v_a^\top \tanh(W_a h_i^{enc} + U_a s_{t-1}^{dec}), \quad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \quad c_t = \sum_i \alpha_{t,i} h_i^{enc} \quad (2)$$

Les coefficients d'attention  $\alpha_{t,i}$  permettent de pondérer dynamiquement les instants passés en fonction de leur pertinence prédictive. Ce mécanisme est particulièrement utile pour détecter des événements transitoires critiques tels que des micro-rafales de trafic ou des congestions soudaines.

**Décodeur :** Le décodeur génère la trajectoire prédite sur un horizon de 15 secondes pour les trois indicateurs clés de performance considérés : le débit, la latence et la perte de paquets.

Le processus de génération est autorégressif : à chaque itération, le décodeur reçoit en entrée sa propre sortie précédente concaténée avec le vecteur de contexte  $c_t$  fourni par le mécanisme d'attention. Cette stratégie permet de modéliser explicitement les dépendances temporelles entre les prédictions successives.

**Capacité du modèle :** L'ensemble de l'architecture totalise **5 920 643 paramètres entraînables**. Cette capacité permet de modéliser efficacement les dynamiques non linéaires complexes observées dans les différentes tranches réseau (eMBB, URLLC et mMTC), tout en restant compatible avec les contraintes temporelles du système de contrôle.

### 5.1.3 Entraînement

L'entraînement vise à stabiliser la convergence et à gérer la nature distribuée des données. L'entraînement local de chaque modèle LSTM utilise l'optimiseur **AdamW** avec un taux d'apprentissage initial de  $10^{-3}$  et un coefficient de régularisation L2 (*weight decay*) de  $10^{-4}$ . La fonction objectif à minimiser est la **HuberLoss**, qui combine la robustesse aux valeurs aberrantes de l'erreur absolue moyenne (L1) avec la stabilité de convergence de l'erreur quadratique (L2) :

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } |y - \hat{y}| \leq \delta \\ \delta (|y - \hat{y}| - \frac{1}{2}\delta) & \text{sinon} \end{cases} \quad (3)$$

Un ordonnanceur de taux d'apprentissage **ReduceLROnPlateau** divise le pas d'apprentissage par 2 en cas de stagnation de la perte de validation sur 5 époques consécutives.

Afin d'accélérer l'apprentissage initial du décodeur, une stratégie de *Teacher Forcing* linéaire décroissante est appliquée. Le principe de cette technique est d'éviter l'accumulation d'erreurs lors des premières phases de l'entraînement en guidant ponctuellement le modèle sur la bonne trajectoire. La probabilité de fournir la véritable valeur cible au décodeur (plutôt que sa propre prédiction) commence à 0,5 et diminue proportionnellement à l'avancement des époques :

$$P = \max\left(0, 0,5 - \frac{\text{epoch}}{\text{epochs\_totales}}\right) \quad (4)$$

Un split temporel 80/20 a été réalisé afin d’entraîner et d’évaluer le modèle. Ainsi, pour chaque scénario simulé, les premiers 80 % de la durée totale sont dédiés à l’entraînement, tandis que les 20 % restants sont réservés au test.

L’entraînement pour chaque slice se déroule en quatre étapes :

1. **Entraînement local par gNB** : Chaque gNB (Macro, Commerce, Industrie) entraîne son propre modèle LSTM sur *ses données exclusivement* durant 50 époques. Le fait de restreindre l’accès de chaque modèle local à son propre gNB — sans voir les données des autres sites — est la procédure dite *LoGO* (Local-only Gradient Optimization). Les distributions non-IID entre gNBs conduisent à des modèles locaux sensiblement différents.
2. **FedAvg** : Les poids des 3 modèles locaux sont moyennés afin de produire un modèle global agrégé. Il ne s’agit pas d’une moyenne des prédictions mais bien d’une moyenne des paramètres internes :  $w_{global} = \frac{1}{|G|} \sum_{g \in G} w_g$ . Ce modèle global constitue un point de départ robuste, synthétisant les spécificités de chaque gNB sans avoir été exposé à leurs données brutes.
3. **FedProx — affinage global** : Le modèle global est réentraîné sur *toutes les données combinées* (jusqu’à 200 époques, early stopping patience = 30) avec le terme proximal FedProx ( $\mu = 0,01$ ) pour rester proche de l’agrégat et éviter l’oubli catastrophique des particularités locales :  $\mathcal{L}_{FedProx} = \mathcal{L}_{local} + \frac{\mu}{2} \|w - w_{global}\|^2$ . Les métriques F1/Recall/Precision reportées dans le tableau 3 sont évaluées sur ce modèle.
4. **Modèles finaux** : Un modèle final est entraîné pour chaque slice, initialisé depuis l’agrégat FedAvg puis affiné sur l’intégralité du dataset sans terme proximal. Ce sont ces modèles (`model_final_{slice}.pt`) qui sont utilisés dans la boucle fermée (notebooks 03 et 04).

| Slice          | F1           | Précision    | Recall       | MAE latence |
|----------------|--------------|--------------|--------------|-------------|
| eMBB           | 0,866        | 0,764        | 1,000        | 43,8 ms     |
| URLLC          | 0,771        | 1,000        | 0,628        | 28,7 ms     |
| mMTC           | 0,645        | 1,000        | 0,476        | 0,07 ms     |
| <b>Moyenne</b> | <b>0,761</b> | <b>0,921</b> | <b>0,701</b> | —           |

TABLE 3 – Résultats de classification SLA du modèle LSTM Seq2Seq sur 762 fenêtres de test.

Plusieurs conclusions peuvent être tirées de ces résultats :

- Pour eMBB, le Recall de 1,000 démontre qu’aucune violation n’est manquée, mais la précision plus faible (0,764) montre que le modèle produit des fausses alarmes qui restent acceptables dans notre contexte : un excès de précaution entraîne une sur-allocation momentanée, sans violation de SLA.
- Pour URLLC, la précision parfaite (1,000) montre que toutes les alertes émises sont réelles, mais qu’environ 37% des violations passent inaperçues (Recall = 0,628). Cela indique que l’URLLC est la slice la plus difficile à prédire car ses pics de latence sont souvent brutaux et courts (1 à 2 secondes), insuffisamment visibles dans une fenêtre d’observation de 60 secondes.
- Pour mMTC, le F1 Score est plus faible (0,645) sans pour autant être critique, ce slice violant rarement ses SLA (seuil très souple à 300 ms).

## 5.2 Réallocation des ressources

Le problème de réallocation proactive est formulé comme un programme linéaire en nombres entiers mixtes (MILP). Pour chaque gNB  $g$  et chaque fenêtre temporelle, le MILP sélectionne le nombre de RBs optimal à allouer à chacune des 3 slices de façon à minimiser conjointement la dégradation du débit et la probabilité de violation de SLA.

Les paramètres réseau fixes sont récapitulés ci-dessous. La répartition par défaut des RBs et les capacités de débit par RB (calibrées empiriquement sur le scénario NormalLoad,  $\mu^{g,s} = tp_{normal}^{g,s} / R_{def}^{g,s}$ )

sont :

$$R_{def} = \begin{pmatrix} & \text{eMBB} & \text{URLLC} & \text{mMTC} \\ \text{Macro} & 20 & 17 & 13 \\ \text{Commerce} & 21 & 11 & 3 \\ \text{Industrie} & 6 & 12 & 7 \end{pmatrix} \quad \mu \text{ (Mbps/RB)} = \begin{pmatrix} & \text{eMBB} & \text{URLLC} & \text{mMTC} \\ \text{Macro} & 0,560 & 0,125 & 0,050 \\ \text{Commerce} & 0,229 & 0,097 & 0,050 \\ \text{Industrie} & 0,400 & 0,195 & 0,050 \end{pmatrix} \quad (5)$$

### 5.2.1 Signal de violation SLA : deux approches selon le contexte

La mesure du packet loss dans Simu5G diffère selon que l'on opère en mode offline (données pré-enregistrées) ou en boucle live avec le contrôleur C++ modifié.

**Contexte offline — notebooks nb02/03/04.** Dans les données brutes générées par Simu5G et stockées dans `all_simu5g_trans.csv`, le mécanisme de buffering MAC maintient le taux de perte natif (`Slice_Packet_Loss_pct`) à 0% même en surcharge : les paquets s'accumulent en file d'attente et sont livrés avec une latence croissante, sans jamais être drop. Cette variable est donc inexploitable comme signal de congestion.

Pour contourner cette contrainte, nous définissons un **indicateur binaire de violation SLA** basé sur la latence :

$$b_t^{g,s} = \mathbf{1}[\text{latence}_t^{g,s} > L_{max}^s] \in \{0, 1\} \quad (6)$$

La moyenne sur une fenêtre donne le taux de violation observé :

$$p^{g,s} = \frac{1}{T} \sum_{t=1}^T b_t^{g,s} \quad (7)$$

C'est cette variable  $p$  qui alimente la calibration de la sigmoïde et l'estimation de  $\rho$  dans tout le pipeline offline.

**Contexte live — contrôleur C++ modifié.** Pour la validation en boucle réelle, nous avons modifié le module `SliceController.cc` de Simu5G afin d'exposer un **vrai taux de perte paquet**, calculé à partir des compteurs applicatifs envoyés côté UE (`cbrSentPkt`) et reçus côté serveur (`cbrRcvdPkt`) :

$$pkt\_loss^{g,s} = \max\left(0, 1 - \frac{rcvdPkts^{g,s}}{sentPkts^{g,s}}\right) \times 100 \quad (8)$$

Cette métrique, écrite toutes les secondes dans `metrics_live.csv`, est ensuite lue directement par le contrôleur Python comme troisième feature d'entrée du LSTM. Contrairement à la loss MAC (qui reste à 0%), ce taux capture les pertes applicatives réelles dues à la congestion — notamment les abandons de connexion TCP et les timeouts UDP en situation de surcharge sévère.

### 5.2.2 Définition et estimation du taux de charge $\rho$

La variable centrale du moteur d'allocation est le **taux de charge**  $\rho^{g,s}$ , défini comme le rapport entre le trafic offert (demande)  $\lambda^{g,s}$  et la capacité de la slice à l'allocation par défaut  $cap^{g,s}$  :

$$\rho^{g,s} = \frac{\lambda^{g,s}}{cap^{g,s}}, \quad cap^{g,s} = R_{def}^{g,s} \times \mu^{g,s} \quad (9)$$

$\rho < 1$  indique un réseau sous-chargé ;  $\rho > 1$  signifie que la demande dépasse la capacité. La demande  $\lambda$  n'est pas directement mesurable : on la reconstruit depuis le débit observé  $tp^{g,s}$  et le taux de violation  $p^{g,s}$ , selon trois cas :

$$\rho^{g,s} = \begin{cases} 2,0 & \text{si } p^{g,s} \geq 0,99 \\ \min\left(\frac{tp^{g,s}}{cap^{g,s} \times (1 - p^{g,s})}, 3,0\right) & \text{si } 0,01 < p^{g,s} < 0,99 \\ \frac{tp^{g,s}}{cap^{g,s}} & \text{si } p^{g,s} \leq 0,01 \end{cases} \quad (10)$$

### Justification des trois cas.

- $p \leq 0,01$  — **charge normale** : aucune violation SLA ; le réseau livre la quasi-totalité du trafic offert. On a  $tp \approx \lambda$ , d'où  $\rho = tp/cap$ .
- $0,01 < p < 0,99$  — **surcharge partielle** : le buffer absorbe la congestion mais génère de la latence. Le débit mesuré correspond aux paquets livrés dans les délais SLA :  $tp \approx \lambda \times (1 - p)$ . En inversant :  $\lambda = tp/(1 - p)$ , d'où  $\rho = tp/(cap \times (1 - p))$ .
- $p \geq 0,99$  — **saturation totale** : toute la capacité est épuisée,  $\lambda \gg cap$ . On fixe  $\rho = 2,0$  comme valeur sentinelle de saturation extrême.

### 5.2.3 Calibration de la sigmoïde $p_{loss}(\rho)$

La probabilité de violation SLA est modélisée par une sigmoïde calibrée sur les données Simu5G, indépendamment pour chaque couple  $(g, s)$  :

$$p_{loss}(\rho; k^{g,s}, \rho_c^{g,s}) = \frac{1}{1 + \exp(-k^{g,s}(\rho - \rho_c^{g,s}))} \quad (11)$$

Les paramètres  $(k^{g,s}, \rho_c^{g,s})$  sont ajustés par régression non-linéaire (`scipy.optimize.curve_fit`) sur les paires  $(\rho_i^{g,s}, p_i^{g,s})$  collectées par scénario via l'équation (10). La sigmoïde fait le lien entre un taux de charge et une probabilité de violation :

- $\rho_c^{g,s}$  est le point d'inflexion : valeur de  $\rho$  où  $p_{loss} = 0,5$ .
- $k^{g,s}$  est la pente de la transition. Une valeur élevée ( $k = 15,27$  pour URLLC-Commerce) rend le coût très sensible aux erreurs d'estimation de  $\hat{\rho}$  autour du seuil, ce qui explique le gain modéré du MILP sur ce site.

La fonction inverse `inv_sigmoid` permet de retrouver  $\rho$  depuis une probabilité observée :

$$\text{inv\_sigmoid}(p, k, \rho_c) = \rho_c - \frac{1}{k} \ln\left(\frac{1}{p} - 1\right) \quad (12)$$

### 5.2.4 Estimation de $\hat{\rho}$ dans la boucle fermée

Dans la boucle fermée,  $\rho$  n'est plus estimé depuis des observations mais **prédit** depuis la sortie du LSTM. Le modèle génère une séquence de 15 pas de temps prévisionnels  $(tp_t, lat_t^{log1p}, loss_t)$ . La latence est récupérée en espace original par  $lat_{ms,t} = \exp(lat_t^{log1p}) - 1$ , puis on calcule la fraction prédite de pas en violation SLA :

$$\hat{p}^{g,s} = \frac{1}{15} \sum_{t=1}^{15} \mathbf{1}[lat_{ms,t} > L_{max}^s] \quad (13)$$

La charge prédite  $\hat{\rho}$  est déduite de  $\hat{p}$  en appliquant le même schéma en trois cas, mais en utilisant l'inverse de la sigmoïde calibrée pour le cas intermédiaire :

$$\hat{\rho}^{g,s} = \begin{cases} 2,0 & \text{si } \hat{p} \geq 0,99 \\ \text{inv\_sigmoid}(\hat{p}, k^{g,s}, \rho_c^{g,s}) & \text{si } 0,01 < \hat{p} < 0,99 \\ \hat{tp}/cap^{g,s} & \text{si } \hat{p} \leq 0,01 \end{cases} \quad (14)$$

Cette formulation est cohérente : lors de la calibration on a estimé  $\rho$  depuis  $p$  via la formule (10) ; ici on fait l'opération symétrique depuis une  $\hat{p}$  prédite.

### 5.2.5 Fonctions de coût et formulation MILP

Une fois  $\hat{\rho}^{g,s}$  connu, on précalcule pour chaque slice  $s$  et chaque candidat  $k \in \{1, \dots, R_{max}^g\}$  le coût associé à l'allocation de  $k$  RBs.

**Charge résiduelle sous  $k$  RBs.** Allouer  $k$  RBs au lieu de  $R_{def}^{g,s}$  modifie la charge selon :

$$\hat{\rho}_k^{g,s} = \hat{\rho}^{g,s} \times \frac{R_{def}^{g,s}}{k} \quad (15)$$

Plus  $k$  est grand, plus la charge résiduelle diminue, et moins la probabilité de violation  $p_{loss}(\hat{\rho}_k)$  est élevée.

**Débit effectivement livré.** La demande estimée est  $\lambda^{g,s} = \hat{\rho}^{g,s} \times cap^{g,s}$ . Sous l'allocation  $k$  :

$$cap_k^{g,s} = k \times \mu^{g,s} \quad (16)$$

$$T_s^{g,s}(k) = \min(\lambda^{g,s}, cap_k^{g,s}) \times (1 - p_{loss}(\hat{\rho}_k^{g,s})) \quad (17)$$

$\min(\lambda, cap_k)$  plafonne la demande à la capacité disponible ; le facteur  $(1 - p_{loss})$  représente la fraction délivrée sans violation SLA.

**Pénalités.** La pénalité de débit et la pénalité de perte sont :

$$p_{tp}^{g,s}(k) = \max\left(0, 1 - \frac{T_s^{g,s}(k)}{T_{min}^s}\right) \cdot \mathbf{1}[\lambda^{g,s} \geq T_{min}^s] \quad (18)$$

$$p_{loss}^{g,s}(k) = p_{loss}(\hat{\rho}_k^{g,s}) \quad (19)$$

L'indicateur  $\mathbf{1}[\lambda \geq T_{min}]$  évite de pénaliser les slices dont la demande est inférieure à l'exigence SLA (cas mMTC à faible charge). La **fonction de coût combinée** pour le candidat  $(s, k)$  est :

$$c^{g,s}(k) = 0,6 \cdot p_{tp}^{g,s}(k) + 0,4 \cdot p_{loss}^{g,s}(k) \quad (20)$$

**Programme d'optimisation.** Les variables de décision sont les binaires  $x_{s,k} \in \{0, 1\}$ , où  $x_{s,k} = 1$  signifie que la slice  $s$  reçoit  $k$  RBs. Le MILP s'écrit :

$$\min_{\{x_{s,k}\}} F = \sum_{s \in S} \sum_{k=1}^{R_{max}^g} c^{g,s}(k) \cdot x_{s,k} \quad (21)$$

sous les contraintes :

$$\sum_{k=1}^{R_{max}^g} x_{s,k} = 1, \quad \forall s \in S \quad (\text{exactement un } k \text{ choisi par slice}) \quad (22)$$

$$\sum_{s \in S} \sum_{k=1}^{R_{max}^g} k \cdot x_{s,k} = R_{max}^g \quad (\text{budget totalement utilisé}) \quad (23)$$

$$x_{s,k} \in \{0, 1\}, \quad \forall (s, k) \quad (24)$$

La contrainte de budget est une **égalité stricte** : tous les RBs sont toujours assignés. En sortie, l'allocation optimale est  $R^{*,g,s} = \sum_k k \cdot x_{s,k}^*$ . Le MILP est résolu par `scipy.optimize.milp` en **4,87 ms** en moyenne par gNB (0,38 s pour 78 problèmes dans nb02), garantissant une intégration temps réel dans la boucle de contrôle à horizon 15 s.

## 6 Résultats

L'évaluation de notre système est structurée en deux temps complémentaires. Un banc d'essai *offline* sur l'intégralité du jeu de données de test permet d'abord de mesurer le gain d'optimisation de la fonction de coût  $F(R)$  de manière exhaustive et reproductible. Une validation *live* en boucle réelle sur le simulateur Simu5G vient ensuite confirmer ces résultats dans un environnement dynamique, où les décisions d'allocation ont un effet causal immédiat sur les métriques réseau mesurées.

### 6.1 Protocole d'évaluation offline

L'ensemble des évaluations *offline* s'appuie sur une partition temporelle stricte du dataset `all_simu5g_trans.csv` (255 812 lignes, 26 scénarios). Afin d'éviter toute fuite de données (*data leakage*), le split 80/20 est réalisé sur le plan temporel au sein de chaque scénario : les premiers 80 % de la durée de chaque série servent à l'entraînement du LSTM, tandis que les 20 % restants constituent le jeu de test. Après construction des fenêtres glissantes ( $T_{obs} = 60$  s,  $T_{pred} = 15$  s, pas = 1 s), ce protocole produit **762 fenêtres de test** par slice, sur lesquelles sont mesurés les gains d'allocation.

La métrique principale est la réduction relative de la fonction de coût  $F(R)$  entre l'allocation statique de référence  $R_{def}$  et l'allocation optimisée  $R^*$  calculée par le MILP :

$$\text{Gain}(\%) = \frac{F(R_{def}) - F(R^*)}{F(R_{def})} \times 100 \quad (25)$$

Quatre configurations sont comparées :

- **Baseline statique** : allocation  $R_{def}$  fixe, sans optimisation.
- **MILP seul — oracle** : le MILP reçoit les KPIs *observés* à l'instant  $t$ . Borne inférieure d'un système sans anticipation.
- **LSTM + MILP — boucle fermée** : le MILP est alimenté par les KPIs *prédits* à  $t + 15$  s. Notre contribution principale.
- **LSTM + MILP + apprentissage en ligne** : le LSTM est mis à jour de manière incrémentale sur chaque fenêtre de test, avec agrégation FedAvg toutes les 5 fenêtres pour eMBB.

## 6.2 Résultats offline

### 6.2.1 Vue d'ensemble comparative

| Configuration                    | Gain moyen $F(R)$ | Apport vs. MILP seul |
|----------------------------------|-------------------|----------------------|
| Baseline statique                | 0%                | —                    |
| MILP seul — oracle               | 20,7%             | référence            |
| LSTM + MILP (boucle fermée)      | <b>39,7%</b>      | +19,0 pp             |
| LSTM + MILP + OL (global)        | 37,2%             | +16,5 pp             |
| LSTM + MILP + OL (régime établi) | <b>40,8%</b>      | +20,1 pp             |

TABLE 4 – Gain moyen de réduction de  $F(R)$  par rapport à l'allocation statique, évalué sur 762 fenêtres de test (26 scénarios, 3 gNBs).

Le premier résultat marquant est l'écart entre le MILP seul (20,7%) et le MILP alimenté par le LSTM (39,7%). Ce delta de +19 **points de pourcentage** quantifie précisément la valeur ajoutée de l'anticipation temporelle : un solveur exact qui opère sur des métriques observées à l'instant présent ne peut allouer les ressources *avant* que la surcharge ne se matérialise.

### 6.2.2 Analyse par station de base

| gNB           | Budget (RBs) | MILP oracle  | LSTM+MILP    | LSTM+MILP+OL |
|---------------|--------------|--------------|--------------|--------------|
| Macro         | 50           | 20,7%        | 35,0%        | 32,5%        |
| Commerce      | 35           | 15,8%        | 14,9%        | 28,3%        |
| Industrie     | 25           | 25,1%        | <b>62,1%</b> | 48,3%        |
| <b>Global</b> | —            | <b>20,7%</b> | <b>39,7%</b> | <b>37,2%</b> |

TABLE 5 – Gain de réduction de  $F(R)$  par gNB sur 762 fenêtres de test.

**gNB Industrie — gain exceptionnel (62,1%).** Ce site présente le plus petit budget radio (25 RBs) et des profils de trafic fortement contrastés entre scénarios calmes et saturés. L'allocation statique conduit à une fonction de coût très élevée sous charge. Le LSTM, en anticipant ces montées en charge, permet au MILP de réallouer de façon optimale bien avant la congestion. L'élasticité de la marge d'amélioration est maximale à Industrie, où chaque RB supplémentaire alloué préventivement a un impact immédiat sur la réduction des pénalités SLA.

**gNB Macro — gain solide (35,0%).** Le site Macro héberge une population d'utilisateurs dense. Sa fonction de coût sous allocation statique est élevée ( $F_{def} = 1,799$  en médiane), et le MILP converge vers  $F^* = 1,099$ , soit un gain de 35%. Le plafond de gain est ici partiellement structurel : la saturation physique de la bande passante macro en scénarios extrêmes limite la marge d'amélioration atteignable par toute réallocation.

**gNB Commerce — gain modéré (14,9%), justifié par la sigmoïde.** Ce résultat s'explique par la pente très abrupte de la sigmoïde URLLC calibrée sur ce site ( $k = 15,27$ ) : de petites erreurs de prédiction de  $\hat{\rho}$  produisent de fortes variations de pénalité, conduisant le MILP à des allocations conservatrices. L'apprentissage en ligne remédie partiellement à ce problème : en adaptant le LSTM aux dynamiques locales du Commerce, le gain passe de 14,9% à **28,3%**, soit +13,4 points.

### 6.3 Apprentissage en continu

Le mécanisme d'apprentissage en continu implémente une mise à jour à chaque séquence traitée du LSTM sur les 762 fenêtres de test. À chaque fenêtre, les erreurs de prédiction réelles sont stockées dans un *replay buffer* (taille 64), et le modèle est mis à jour par mini-batch gradient descent (Adam,  $\eta = 10^{-5}$ ). Pour eMBB, une agrégation FedAvg est appliquée toutes les 5 fenêtres.

| Phase                                   | Gain $F(R)$  | MAE prédiction | Train loss   |
|-----------------------------------------|--------------|----------------|--------------|
| Warmup (1 <sup>ère</sup> moitié)        | 34,1%        | 0,0360         | 0,204        |
| Régime établi (2 <sup>ème</sup> moitié) | <b>40,3%</b> | <b>0,0251</b>  | <b>0,152</b> |
| Global Apprentissage continue           | 37,2%        | 0,0306         | —            |

TABLE 6 – Performance de l'apprentissage en ligne par phase (762 fenêtres, split 50/50). Train loss globale : 0,183  $\rightarrow$  0,006 ( $\times 33$  de réduction).

Le gain global observé avec l'apprentissage continu (37,2%) demeure inférieur à celui obtenu avec l'approche sans apprentissage continu (39,7%) alors même que son régime établi (40,3%) lui est supérieur. Ce paradoxe s'explique entièrement par la phase de chauffe : les premières fenêtres de test servent à calibrer le modèle sur la distribution réelle, dégradant temporairement les performances. En production, une pré-initialisation du *replay buffer* sur les données récentes supprimerait cette pénalité initiale.

La divergence L2 post-FedAvg pour eMBB (0,07) confirme la cohérence de l'agrégation fédérée. Les divergences plus élevées pour URLLC (17,0) et mMTC (17,6) reflètent leurs distributions de trafic fortement hétérogènes entre gNBs — cohérent avec la distribution non-IID du dataset.

### 6.4 Validation live sur Simu5G

La validation en boucle réelle déploie notre contrôleur sur le scénario **ControllerDemo\_v2** : 300 secondes de simulation, 24 UEs répartis sur les trois gNBs, soumis à des rafales de trafic imprévisibles. Il est important de souligner que ce scénario a été méticuleusement dimensionné pour ne pas être structurellement invalide. En effet, soumettre le réseau à une charge physiquement impossible à écouler (par exemple, concentrer virtuellement un million d'utilisateurs sur une unique station de base rurale) mènerait inévitablement à un effondrement du réseau, rendant toute tentative d'optimisation mathématique de l'allocation parfaitement vaine. Le scénario choisi garantit ainsi que les dégradations observées sont théoriquement solvables par une gestion intelligente des ressources radio. Par ailleurs, cette évaluation propose un focus particulier sur la tranche URLLC, afin d'éprouver la capacité du système à garantir ses exigences temporelles critiques face aux pics soudains. Trois modes sont comparés :

Le résultat le plus révélateur est l'échec total du MILP réactif ( $\Delta = 0,0$  pp). Ce résultat n'est pas une anomalie : il démontre structurellement pourquoi les approches réactives sont inadaptées aux services URLLC. Les violations de cette tranche sont causées par des micro-rafales de trafic d'une durée de 1 à 2 secondes. Un contrôleur qui observe la surcharge *après* son apparition intervient systématiquement *après* que la violation s'est produite. Notre contrôleur LSTM + MILP, en anticipant l'état du réseau à  $t + 15$  s, pré-alloue les ressources URLLC *avant* que les micro-rafales ne se manifestent.



| Mode                           | Violations URLLC (%) | Gain vs. baseline |
|--------------------------------|----------------------|-------------------|
| Baseline (allocation statique) | 32,4%                | —                 |
| MILP périodique (réactif)      | 32,4%                | +0,0 pp           |
| <b>LSTM + MILP (adaptatif)</b> | <b>26,9%</b>         | <b>−5,6 pp</b>    |

TABLE 7 – Taux de violations SLA URLLC mesuré en simulation live (ControllerDemo\_v2,  $t = 300$  s, 24 UEs).

La réduction de **5,6** points de pourcentage de violations URLLC signifie que **17%** de la totalité des violations SLA URLLC qui survenaient ont été totalement éliminées. Cela confirme l’avantage de l’anticipation pour la réallocation.

Enfin, il convient de souligner que ces résultats, sont obtenus à l’échelle d’une infrastructure restreinte à trois stations de base. À l’échelle d’une région entière, regroupant des centaines d’antennes, la granularité et la diversité spatiale du trafic offrirait une base de ressources plus grande qui permettrait une réallocation plus efficace dans les zones demandeuse de ressources.

## 7 Conclusion et perspectives

### 7.1 Synthèse des contributions

Ce travail a proposé et validé une architecture de contrôle en boucle fermée pour le *network slicing* 5G, visant à coupler de manière cohérente un modèle prédictif des indicateurs de performance réseau avec un moteur d’allocation de ressources respectant explicitement les contraintes de qualité de service (SLA). Cette approche répond à une limite fréquemment observée dans l’état de l’art, où les mécanismes de prédiction et d’allocation restent généralement dissociés.

La conception de l’architecture repose sur la combinaison de trois approches complémentaires adaptées aux contraintes du problème étudié. Un modèle LSTM Seq2Seq a été retenu pour la prédiction des KPIs réseau, en raison de son efficacité reconnue sur des séries temporelles à horizon court. Contrairement à des architectures plus complexes telles que les Transformers, ce type de modèle présente un compromis favorable entre précision prédictive et coût computationnel, particulièrement adapté à une anticipation à court terme (+15 s). L’intégration d’un mécanisme d’attention permet en outre d’améliorer la prise en compte des instants historiques les plus informatifs, ce qui favorise la détection précoce des variations rapides du trafic.

Afin de prendre en compte l’hétérogénéité des environnements réseau entre stations de base, une stratégie d’apprentissage fédéré a été adoptée. Cette approche permet d’entraîner des modèles locaux sur des données spécifiques à chaque gNB, tout en maintenant une cohérence globale grâce à l’agrégation des paramètres. Ce choix répond aux contraintes de distribution non-IID des données réseau et limite les besoins en transfert de données, ce qui constitue un avantage en termes de scalabilité.

Enfin, l’allocation des ressources radio a été formulée sous la forme d’un programme linéaire en nombres entiers mixtes (MILP). Ce choix permet d’intégrer explicitement les contraintes SLA dans le processus décisionnel et d’obtenir des solutions optimales sous contraintes formelles. Contrairement à certaines approches basées sur l’apprentissage par renforcement, la formulation MILP fournit des décisions déterministes et interprétables, avec un temps de résolution compatible avec les contraintes temps réel observées expérimentalement.

Ces choix méthodologiques ont conduit au développement des contributions suivantes.

#### **Contribution 1 — Modèle prédictif Seq2Seq fédéré**

Un modèle BiLSTM Seq2Seq intégrant un mécanisme d’attention a été développé afin de prédire l’évolution des KPIs réseau à un horizon de 15 secondes. L’entraînement repose sur une procédure fédérée combinant apprentissage local par gNB, agrégation FedAvg, régularisation FedProx ( $\mu = 0,01$ ) et phase d’affinage finale. Cette approche permet de tenir compte de l’hétérogénéité des environnements réseau. Le modèle atteint un score F1 moyen de 0,761 et une précision moyenne de 0,921 pour la détection des violations SLA.

#### **Contribution 2 — Formulation MILP intégrant les contraintes SLA**

Le problème d’allocation des ressources a été formulé comme un programme linéaire en nombres entiers mixtes minimisant une fonction de coût combinant pénalité de débit et probabilité de violation SLA.

Une calibration empirique des fonctions sigmoïdes  $p_{loss}(\rho)$  permet d'intégrer explicitement la dynamique du réseau dans le modèle d'optimisation. Le solveur atteint un temps moyen de résolution inférieur à 5 ms par fenêtre.

#### **Contribution 3 — Intégration en boucle fermée**

Le couplage entre le modèle prédictif et le solveur d'optimisation permet une allocation proactive des ressources. Cette stratégie produit un gain global de 39,7% par rapport à une approche réactive basée uniquement sur le MILP. L'ajout d'un mécanisme d'apprentissage incrémental permet d'atteindre un gain de 40,8% après convergence.

#### **Contribution 4 — Validation expérimentale**

L'architecture proposée a été validée dans un environnement simulé basé sur Simu5G. Les résultats montrent une réduction mesurable des violations SLA, notamment pour la slice URLLC, illustrant l'intérêt d'une approche anticipative dans des environnements réseau dynamiques.

## **7.2 Limites et discussion critique**

**1 — Données exclusivement synthétiques.** L'ensemble de notre pipeline repose sur des données générées par Simu5G. Si ce simulateur reproduit fidèlement les mécanismes de la pile protocolaire 5G, ses modèles de propagation radio et ses profils de trafic utilisateurs restent des approximations. L'accès à des traces réelles d'un opérateur télécoms — non disponibles publiquement pour des raisons de confidentialité industrielle — permettrait de valider la généralisabilité de notre approche. Comme le relève la littérature [5], les datasets réels présentent une variabilité structurellement différente des scénarios simulés.

**2 — Contrainte physique de Simu5G : la *backpressure*.** Le mécanisme de buffering MAC de Simu5G maintient le taux de perte natif (`Slice_Packet_Loss_pct`) à 0% même en surcharge. Dans le pipeline offline (nb02/03/04), nous contournons cette limitation par un indicateur binaire de dépassement de latence ( $b_t = \mathbf{1}[\text{latence} > L_{max}]$ ), qui sert de proxy à la congestion. Pour la validation live, nous avons modifié le module `SliceController.cc` en souscrivant aux signaux `cbrSentPkt` (UEs) et `cbrRcvdPkt` (serveurs), permettant de calculer un vrai taux de perte applicatif écrit dans `metrics_live.csv` et exploité directement par le contrôleur Python. Cette distinction introduit une légère hétérogénéité entre les features d'entraînement (latence proxy) et les features live (perte réelle), dont l'impact est atténué par la normalisation commune des scalars.

**3 — Contrainte d'échelle : réseau à 3 gNBs.** La simulation complète de notre architecture nécessite des ressources de calcul et de stockage significatives. L'extension à un réseau de taille opérationnelle (plusieurs dizaines de gNBs) est hors de portée du matériel disponible pour ce projet. Bien que l'architecture fédérée soit théoriquement extensible, la procédure de calibration sigmoïde du MILP nécessiterait une ré-exécution sur chaque nouveau site.

**4 — Recall URLLC (0,628) : limite inhérente au trafic bursty.** Notre modèle manque environ 37% des violations URLLC. Cette limitation reflète la nature intrinsèquement imprévisible des micro-rafales : des pics de 1 à 2 secondes dans une fenêtre de 60 secondes représentent un signal extrêmement faible. La précision parfaite (1,000) confirme que les alertes émises sont correctes ; le problème est que certaines violations surviennent sans signe avant-coureur détectable sur l'horizon d'observation.

**5 — Écart offline/live pour le gNB Macro.** Dans la validation live, la contention MAC générée par 24 UEs introduit une latence d'application de l'allocation non modélisée dans le framework *offline*. Un modèle de délai d'application intégré dans la formulation MILP permettrait de réduire cet écart.

## **7.3 Perspectives**

**Fonction de perte asymétrique.** Remplacer la HuberLoss symétrique par une *wMAE* [13] pénalisant davantage les sous-estimations de charge améliorerait le Recall URLLC (0,628) au prix d'un léger surcroît de fausses alarmes — compromis favorable pour un service à contraintes strictes.

**Modèle URLLC spécialisé.** Un modèle dédié entraîné sur des fenêtres plus courtes (résolution 100 ms au lieu de 1 s) pourrait capturer les micro-rafales URLLC actuellement invisibles à notre LSTM à horizon 60 secondes.

**Généralisation avec données réelles.** L'accès à des traces de trafic réelles (à l'image de [5], ou via des partenariats opérateurs) permettrait de valider la robustesse de notre architecture hors simulation. Notre pipeline de calibration sigmoïde est conçu pour être réentraînable sur tout dataset historique.

**Extension multi-opérateur via FedAvg inter-domaines.** Notre architecture fédérée est naturellement extensible à un scénario de partage inter-opérateurs : chaque opérateur entraîne ses modèles locaux sans partager ses données brutes, et l'agrégation FedAvg produit un modèle mutualisé.

**Intégration O-RAN.** Nos contrôleurs sont architecturés comme des boucles de contrôle externes communiquant via socket, une topologie directement mappable sur l'architecture xApp/rApp des interfaces O1/A1 du standard O-RAN. Cette compatibilité conceptuelle constitue une base pour un déploiement sur des plateformes O-RAN open-source (OpenAirInterface, O-DU/O-CU).

## Références

- [1] Yu Abiko, Takato Saito, Daizo Ikeda, Ken Ohta, Tadanori Mizuno, and Hiroshi Mineno. Flexible resource block allocation to multiple slices for radio access network slicing using deep reinforcement learning. *IEEE Access*, 8 :68183–68198, 2020.
- [2] Faezeh Bahramisirat, Mark A. Gregory, and Shuo Li. Dynamic network slicing for resource allocation in 5g/b5g networks : An optimization-based approach. *Ad Hoc Networks*, 186 :104194, 2026.
- [3] Yuekai Cai, Peng Cheng, Zhuo Chen, Wei Xiang, Branka Vucetic, and Yonghui Li. Dynamic resource allocation in network slicing with deep reinforcement learning. 2023.
- [4] Wei-Kun Chen, Ya-Feng Liu, Yu-Hong Dai, and Zhi-Quan Luo. Qos-aware and routing-flexible network slicing for service-oriented networks. *IEEE Transactions on Network and Service Management*, 22(6) :6021–6036, 2025.
- [5] Yong-Hoon Choi, Daegyeom Kim, and Myeongjin Ko. 5g traffic datasets, 2023.
- [6] Venkata Ramana K, Ramesh B, Ravindra Changala, Aditya Sai Srinivas T, Praveen Kumar Kalangi, and Bhavsingh M. Optimizing 6g network slicing with the evonetslice model for dynamic resource allocation and real-time qos management. *International Research Journal of Multidisciplinary Technology*, 6(3) :325–340, May 2024.
- [7] Ziran Min, Ziteng Liu, Zhuangwei Kang, Shuang Zhou, Swapna Gokhale, Shashank Shekhar, Charif Mahmoudi, and Aniruddha Gokhale. A novel 5g digital twin approach for traffic prediction and elastic network slice management. In *2024 16th International Conference on COMMunication Systems NETWORKS (COMSNETS)*, pages 497–505, 2024.
- [8] Ahmad M. Nagib, Hatem Abou-Zeid, and Hossam S. Hassanein. Safeslice : Enabling sla-compliant o-ran slicing via safe deep reinforcement learning. In *2025 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*, pages 1–7, 2025.
- [9] Nikolaos Pavlidis, Vasileios Perifanis, Selim F. Yilmaz, Francesc Wilhelmi, Marco Miozzo, Pavlos S. Efraimidis, Remous-Aris Koutsiamanis, Pavol Mulinka, and Paolo Dini. Federated learning in mobile networks : A comprehensive case study on traffic forecasting. *IEEE Transactions on Sustainable Computing*, 10(3) :576–587, 2025.
- [10] Farhad Rezazadeh, Hatim Chergui, Shuaib Siddiqui, Josep Mangués, Houbing Song, Walid Saad, and Mehdi Bennis. Intelligible protocol learning for resource allocation in 6g o-ran slicing. *IEEE Wireless Communications*, 31(5) :192–199, 2024.
- [11] Sajal Saha, Saikat Das, and Glaucio H.S. Carvalho. ConvLstmtransnet : A hybrid deep learning approach for internet traffic telemetry. In *2024 IEEE Virtual Conference on Communications (VCC)*, pages 1–6, 2024.
- [12] Nguyen Phuc Tran, Oscar Delgado, Brigitte Jaumard, and Fadi Bishay. Ml kpi prediction in 5g and b5g networks. In *2023 Joint European Conference on Networks and Communications 6G Summit (EuCNC/6G Summit)*, pages 502–507, 2023.
- [13] Evren Tuna, Asude Baykal, and Alkan Soysal. Multivariate and multistep mobile traffic prediction with sla constraints : A comparative study. *Ad Hoc Networks*, 163 :103594, 2024.
- [14] Shabana Urooj, Rajesh Arunachalam, Mohamad A. Alawad, Kuldeep Narayan Tripathi, Damodaran Sukumaran, and Poonguzhali Ilango. An effective model for network selection and resource allocation in 5g heterogeneous network using hybrid heuristic-assisted multi-objective function. *Expert Systems with Applications*, 248 :123307, 2024.
- [15] Yang Yang, Yechen He, Binnan Zhao, Celimuge Wu, Zhipeng Gao, and Lanlan Rui. Multirepresentation spatial-temporal graph convolutional networks for network traffic prediction. *IEEE Internet of Things Journal*, 12(13) :23085–23099, 2025.